



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 并发与锁机制

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/4/30-2026/5/5

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
五、实验总结与心得体会.....	15
六、附录：参考资料清单.....	16

Section 1 实验概述

首先使用硬件支持的原子指令来实现自旋锁 SpinLock。接着使用 SpinLock 来实现信号量，最后我们使用 SpinLock 和信号量来给出两个实现线程互斥的解决方案

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++/C 程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）/ vscode
 - 代码编译工具：GCC、g++、nasm、ld（链接器）、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：自旋锁与信号量的实现
- 实验任务 2：生产者-消费者问题
- 实验任务 3：哲学家就餐问题

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：自旋锁与信号量的实现
 - 1.1 利用自旋锁和信号量实现同步：利用指导书中实现的自旋锁和信号量方法，解决“消失的芝士汉堡”问题。
 - 1.2 自实现锁机制：原子指令 xchg 来实现自旋锁并不是唯一的方法。例如，x86 指令中提供了另外一个原子指令 **bts** 和 **lock** 前缀等，这些指令也可以用来实现锁机制。实现一个方式不同的锁机制。最后，尝试实现的锁机制解决“消失的芝士汉堡”问题
- 思路分析：

1.1 首先定义类并实现类的方法：spinlock —— 初始化 + 上锁

(asm_atomic_exchange) + 解锁 / semaphore —— 初始化 +P 操作 +V 操纵 (MESA_WakeUp) , 再在线程函数首尾加上相应的 lock (P) 和 unlock (V) 操作, 最后在第一个线程 (不会退出, 没有返回) 中初始化自旋锁 (信号量) 并且执行线程。

1.2 bolt 若为 0, 则线程可以打开锁进入执行区域; 同时 key 总是为 1, 否则就不会进入循环遇到交换的汇编函数, 于是可以得到代码逻辑: 使用 **lock** 前缀的 **bts** 指令, 判断内存中的值是否为 1, 如果 1 则 CF 设置为 1, 并将内存中的值设置为 1, 否则 cf 为 0, 同时将 cf 的值存入 key, 那么若 bolt 为 0, 则 key 为 0, bolt 改为 1, 实现了互斥的功能

- 实验步骤:

1.1

1.1.1 自旋锁

①定义表示自旋锁的类 SpinLock

```
#ifndef SYNC_H
#define SYNC_H

#include "os_type.h"

class SpinLock
{
private:
    uint32 bolt;
public:
    SpinLock();
    void initialize();
    void lock();
    void unlock();
};

#endif
```

②编写类的方法函数: 全局变量的构造函数在我们的操作系统实验中不会被自动调用。所以在 **使用 SpinLock 的时候**, 我们需要手动调用 **SpinLock::initialize**

```

#include "sync.h"
#include "asm_utils.h"
#include "stdio.h"
#include "os_modules.h"

SpinLock::SpinLock()
{
    initialize(); // 全局变量的构造函数在我们的操作系统实验中不会被自动调用,需要手动调用
}

void SpinLock::initialize()
{
    bolt = 0;
}

void SpinLock::lock()
{
    uint32 key = 1;

    do
    {
        asm_atomic_exchange(&key, &bolt);
        //printf("pid: %d\n", programManager.running->pid);
    } while (key);
}

void SpinLock::unlock()
{
    bolt = 0;
}

```

③实现 `asm_atomic_exchange`: `asm_atomic_exchange` 是一个“原子”指令，保证了两个值在交换的过程中不会被中断，在实现 `asm_atomic_exchange` 的时候作了一个重要的假设——形式参数 `register` 指向的变量不是一个共享变量，满足这个条件才会是原子的

```

asm_atomic_exchange:
    push ebp
    mov ebp, esp
    pushad

    mov ebx, [ebp + 4 * 2]; register
    mov eax, [ebx];
    mov ebx, [ebp + 4 * 3]; memory
    xchg [ebx], eax;
    mov ebx, [ebp + 4 * 2]; memory
    mov [ebx], eax;

    popad
    pop ebp
    ret

```

④在 `setup.cpp` 中创建类对象，并且在线程中使用 `.lock()` 和 `unlock()` 方法：第一个

线程不能返回，挂起线程

```
SpinLock aLock;

int cheese_burger;

void a_mother(void *arg)
{
    aLock.lock();
    int delay = 0;

    printf("txc: start to make cheese burger, there are %d cheese burger now\n", cheese_burger);
    cheese_burger += 10;

    printf("txc: oh, I have to hang clothes out.\n");
    delay = 0xffffffff;
    while (delay)
        --delay;

    printf("txc: Oh, Jesus! There are %d cheese burgers\n", cheese_burger);
    aLock.unlock();
}

void a_naughty_boy(void *arg)
{
    aLock.lock();
    printf("boy : Look what I found!\n");
    // eat all cheese_burgers out secretly
    cheese_burger -= 10;
    // run away as fast as possible
    aLock.unlock();
}
```

1.1.2 信号量：自旋锁会存在忙等待、可能饥饿、可能死锁等问题

①定义信号量类：

▲counter：是否有临界资源可供分配

▲waiting：阻塞队列

```
class Semaphore
{
private:
    uint32 counter;
    List waiting;
    SpinLock semLock;

public:
    Semaphore();
    void initialize(uint32 counter);
    void P();
    void V();
};
#endif
```

②实现类的函数方法：

▲初始化

```

Semaphore::Semaphore()
{
    initialize(0);
}

void Semaphore::initialize(uint32 counter)
{
    this->counter = counter;
    semLock.initialize();
    waiting.initialize();
}

```

▲P 操作：counter 和 waiting 需要互斥访问，所以需要在相应位置加上锁，通过设置当前状态实现线程的阻塞（不会进入就绪队列）

```

void Semaphore::P()
{
    PCB *cur = nullptr;

    while (true)
    {
        semLock.lock();
        if (counter > 0) //对counter进行互斥访问
        {
            --counter;
            semLock.unlock();
            return;
        }

        cur = programManager.running;
        waiting.push_back(&(cur->tagInGeneralList)); //对waiting队列进行互斥访问
        cur->status = ProgramStatus::BLOCKED; //设置线程阻塞态

        semLock.unlock();
        programManager.schedule();
    }
}

```

▲V 操作：释放资源，取出阻塞线程并唤醒（设置 ready 状态）

```

void Semaphore::V()
{
    semLock.lock();
    ++counter;
    if (waiting.size()) //存在阻塞的线程，我们需要将其唤醒
    {
        PCB *program = ListItem2PCB(waiting.front(), tagInGeneralList);
        waiting.pop_front();
        semLock.unlock();
        programManager.MESA_WakeUp(program);
    }
    else
    {
        semLock.unlock();
    }
}

```

▲唤醒：MESA 模型

```
void ProgramManager::MESA_WakeUp(PCB *program) {
    program->status = ProgramStatus::READY;
    //printf("wake up program, pid: %d\n", program->pid);
    readyPrograms.push_front(&(program->tagInGeneralList));
}
```

③在 first_thread 中初始化信号量并且执行两个线程函数：

```
void first_thread(void *arg)
{
    // 第1个线程不可以返回,清屏操作
    stdio.moveCursor(0);
    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }
    stdio.moveCursor(0);

    cheese_burger = 0;
    semaphore.initialize(1);

    programManager.executeThread(a_mother, nullptr, "second thread", 1);
    programManager.executeThread(a_naughty_boy, nullptr, "third thread", 1);

    asm_halt();
}
```

1.2： 使用 LOCK BTS 构成原子的读 - 改 - 写指令

▲BTS： bite test and set， 它会先读取 [ebx] 指向的内存地址的第 0 位， 将其存入进位标志位 CF， 然后无论该位之前是多少， 都将该位置为 1

▲LOCK 前缀： 修饰符， 让其后那条指令在多核 / 多线程下原子化

修改后的 asm_atomic_exchange： 注意在 x86 汇编中， 当指令涉及内存地址且没有寄存器来暗示操作大小时， 你必须显式指定， 否则会报错

```
asm_atomic_exchange:
    push ebp
    mov ebp, esp
    pushad

    xor eax, eax
    mov ebx, [ebp + 4 * 3]; memory
    ;使用lock前缀的bts指令，判断内存中的值是否为0，如果为0则CF设置为1,并将内存中的值设
    lock bts dword [ebx], 0 ;显式指定寄存器大小
    ;把cf的值存入eax, al为低八位
    setc al
    ;将eax的值存入ebx指向的内存中
    mov ebx, [ebp + 4 * 2]; memory
    mov [ebx], eax ;

    popad
    pop ebp
    ret
```

- 实验结果： 1.1 和 1.2 都有如下结果

```
txc: start to make cheese burger, there are 0 cheese burger now
txc: oh, I have to hang clothes out.
txc: Oh, Jesus! There are 10 cheese burgers
boy  : Look what I found!
```

----- 实验任务 2 -----

- 任务要求：生产者-消费者问题

问题场景B：抽烟者与供应商

问题描述：酒馆的吧台坐着3位抽烟者和1位供应商。每位抽烟者会不停地卷烟并抽掉，但是要卷起并抽掉一支烟，抽烟者需要三种材料：烟草，纸和胶水。三位抽烟者中，第1位拥有烟草，第2位有纸，第3位有胶水。供应商会源源不断地提供3种材料，每次他会将两种随机材料组合放在吧台的桌面上，拥有剩下那种材料的抽烟者，会立刻取走材料卷一根烟抽掉它，然后给供应商发一个完成信号，供应商就会放另外两种不同材料组合在桌面上，这样的过程一直重复。

2.1 线程的竞争与冲突：不使用任何实现同步/互斥的工具。因此，不同的线程之间可能会产生竞争/冲突，从而无法达到预期的运行结果。

2.2 利用信号量解决问题：简单描述该问题中各个线程间的互斥关系，并使

用信号量机制实现线程的同步。

● 思路分析：

2.1 选择场景 B，因为三位抽烟者和一个供应商共四个线程可能更简单，最开始尝试如果判断两种材料都存在才开始抽烟，但是发现就算没有实现互斥的工具也不会出现竞争现象（可能是目前线程切换的频率没有撞到冲突发生空隙——还没有取材料材料就更换），后面通过搜索，更改将资源获取逻辑改为分步拾取（即先拿烟草，再拿胶水），人为制造非原子性操作，就可以观察到竞争现象

2.2 使用四个信号量在每个线程函数对应位置加上信号量 P/V 操作即可

SUPPLIER 信号量：用于控制供应商的投放行为。其初值为 1，表示吧台初始为空，供应商可以投放。

- 供应商在投放前执行 P(SUPPLIER)，如果吧台上已有材料，供应商会阻塞等待。
- 当抽烟者抽完烟后，执行 V(SUPPLIER)，通知供应商吧台已空。

SMOKER[1-3] 信号量：代表吧台上出现的特定材料组合。其初值均为 0。

- 抽烟者通过 P(SMOKER) 尝试获取自己缺失的材料。如果没有对应的材料，抽烟者会进入阻塞队列。
- 供应商投放随机组合后，执行对应抽烟者的 V(SMOKER) 操作，唤醒特定的抽烟者。

● 实验步骤：

2.1

①供应者：满足源源不断供应，且在 smoker 消耗材料后才可继续提供供应

```

void supplier(void *arg) {
    int type = 0;
    while(true) {
        if (finish_signal == 1) {
            type = (type + 1) % 3;

            if (type == 0) {
                has_paper = 1; has_glue = 1;
                printf("\nSupplier5242: Put [Paper] and [Glue] on table.\n");
            } else if (type == 1) {
                has_tobacco = 1; has_glue = 1;
                printf("\nSupplier5242: Put [Tobacco] and [Glue] on table.\n");
            } else {
                has_tobacco = 1; has_paper = 1;
                printf("\nSupplier5242: Put [Tobacco] and [Paper] on table.\n");
            }

            finish_signal = 0;
        }
    }
}

```

②smoker（三个类似）：

▲最开始

```

void smoker_1(void *arg) {
    while(true) {
        if (has_paper == 1 && has_glue == 1) {
            race_delay();
            has_paper = 0;
            has_glue = 0;
            printf("Smoker_1: Got Paper & Glue! Smoking...\n");
            race_delay();
            finish_signal = 1;
        }
    }
}

```

但是结果看起来像正常运行

```

Supplier: Put [Tobacco] and [Glue] on table.
Smoker_2: Got Tobacco & Glue! Smoking...

Supplier: Put [Tobacco] and [Paper] on table.
Smoker_3: Got Tobacco & Paper! Smoking...

Supplier: Put [Paper] and [Glue] on table.
Smoker_1: Got Paper & Glue! Smoking...

```

▲后来改用：将资源获取逻辑改为分步拾取（即先拿烟草，再拿胶水），人为制造了非原子性操作，并且引入“内存”状态——TOBACCO_FLAG 和 MATCH_FLAG 以及递减操作 (--) 的不确定性（读取 -> 修改 -> 写回）

```

void smoker_1(void *arg) {
    bool TOBACCO_FLAG = false;
    bool MATCH_FLAG = false;
    while(1){
        //printf("smoker1 thread running!\n");
        if (has_tobacco){
            has_tobacco--;
            TOBACCO_FLAG = true;
            printf("smoker_have_paper: GET TOBACCO!\n");
        }
        if (has_glue){
            has_glue--;
            MATCH_FLAG = true;
            printf("smoker_have_paper: GET MATCH!\n");
        }
        if (TOBACCO_FLAG && MATCH_FLAG){
            printf("smoker_have_paper: I am smoking!\n");
            finish_signal++;
            //printf("waiting for producer!\n");
            TOBACCO_FLAG = false;
            MATCH_FLAG = false;
        }
    }
}

```

2.2

①各个线程间：供应者与抽烟者之间同步约束，三个抽烟者之间互斥(资源争夺)

②SUPPLIER 线程函数：

```

void supplier(void *arg){
    SUPPLIER.initialize(1);
    int rand=0;
    while(1){
        SUPPLIER.P();
        int rand=(rand+1)%3;
        printf("5242 offer matertial%d\n",rand+1);
        if(rand==0){
            SMOKER1.V();
        }else if(rand==1){
            SMOKER2.V();
        }else{
            SMOKER3.V();
        }
    }
}

```

③SMOKER 线程函数：

```

void smoker1(void *arg){
    while(1){
        SMOKER1.P();
        printf("smoker1: I am smoking!\n");
        race_delay();
        SUPPLIER.V();
    }
}

```

- 实验结果展示:

2.1 由于双方都只持有部分材料且无法释放，导致系统逻辑无法闭环（无法进入 smoking 分支），成功呈现了由于竞争导致的资源抢占冲突

```

Supplier5242: Put [Tobacco] and [Glue] on table.
smoker_have_paper: GET TOBACCO!
smoker_have_paper: GET MATCH!
smoker_have_paper: I am smoking!
Supplier5242: Put [Tobacco] and [Paper] on table.
smoker_have_paper: GET TOBACCO!
smoker_have_tobacco: GET TOBACCO!

```

2.2 逻辑正确且抽烟供应的过程可以一直重复

```

5242 offer material3
smoker3: I am smoking!
5242 offer material1
smoker1: I am smoking!
5242 offer material2
smoker2: I am smoking!
5242 offer material3
smoker3: I am smoking!
5242 offer material1
smoker1: I am smoking!
5242 offer material2
smoker2: I am smoking!
5242 offer material3
smoker3: I am smoking!
5242 offer material1
smoker1: I am smoking!
5242 offer material2
smoker2: I am smoking!
5242 offer material3
smoker3: I am smoking!
5242 offer material1
smoker1: I am smoking!
5242 offer material2
smoker2: I am smoking!

```

----- 实验任务 3 -----

● 任务要求：哲学家就餐问题

3.1 简单解决方法：创建多个线程来模拟哲学家就餐的场景。然后需要结合信号量来实现理论教材中给出的关于哲学家就餐的简单解决办法（如下）。注意：. 可以通过输出不同哲学家的状态信息，验证你使用教程的方法确实能解决哲学家就餐问题。

一种简单的解决方法是每只筷子都用一个信号量来表示。一个哲学家通过执行操作 `wait()` 试图获取相应的筷子，他会通过执行操作 `signal()` 以释放相应的筷子。因此，共享数据为

`semaphore chopstick[5];`

27

其中，`chopstick` 的所有元素都初始化为 1。哲学家 i 的结构如图 6-14 所示。

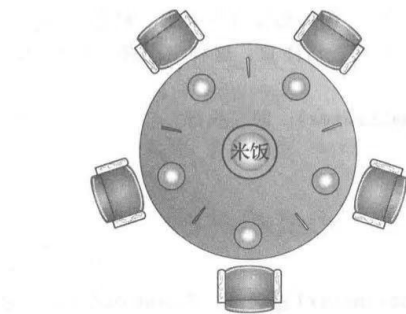


图 6-13 就餐哲学家的情景

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    /* eat for awhile */
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    /* think for awhile */
} while (true);
```

图 6-14 哲学家 i 的结构

3.2 死锁对应策略（策略 3）：使用非对称的解决方案。即规定奇数号的哲学家先拿起他左边的筷子，然后再去拿起他右边的筷子；而偶数号的哲学家则先拿起他右边的筷子，然后再去拿他左边的筷子。

● 思路分析：

3.1 按照图 6-14 编写代码即可，注意在第一个线程中初始化 `semaphore`，否则会因为初始值是 0（或随机值）导致所有哲学家一上来就全部阻塞

3.2 改变不同哲学家拿筷子的先后顺序即可

● 实验步骤：

3.1

```
void pholo0(void *arg){
    int time=0;
    do{
        chop[0].P();
        chop[1].P();

        printf("5242txc is eating: %d\n", ++time);
        race_delay();

        chop[0].V();
        chop[1].V();

        race_delay();
    }while(true);
}
```

WARNING: Image format was not specified for './run/hd.img' and probing guessed raw.

Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.

Specify the 'raw' format explicitly to remove the restrictions.

会出现以上警告，直接诱因可能是增加了全局数组 `chop[5]`，每个哲学家线程的逻辑也变复杂了。

3.2

①由于之前代码逻辑比较简单（只有两行 P 操作），CPU 执行这两行机器码只需要几个纳秒。时钟中断刚好落在两个 P 操作之间的概率极低，所以死锁产生概率不高，所以需要进行**死锁模拟**——每一个哲学家同时拿起左边的筷子，这时筷子没有剩余，产生循环等待，造成死锁

```

void pholo0(void *arg){
    int time=0;
    do{
        chop[0].P();
        race_delay();
        chop[1].P();

        printf("5242txc is eating: %d\n", ++time);
        race_delay();

        chop[0].V();
        chop[1].V();

        race_delay();
    }while(true);
}

```

```

pholo1 is eating: 1
5242txc is eating: 1
pholo4 is eating: 1
pholo3 is eating: 1
pholo2 is eating: 1
pholo1 is eating: 2
5242txc is eating: 2
pholo4 is eating: 2

```

运行一段时间后不再输出，陷入死锁状态

②修改奇数号的哲学家先拿起他右边的筷子，然后再去拿起他左边的筷子

```

void pholo1(void *arg){
    int time=0;
    do{
        chop[2].P();
        race_delay();
        chop[1].P();

        printf("pholo1 is eating: %d\n", ++time);
        race_delay();

        chop[1].V();
        chop[2].V();

        race_delay();
    }while(true);
}

```

```

void pholo3(void *arg){
    int time=0;
    do{
        chop[4].P();
        race_delay();
        chop[3].P();

        printf("pholo3 is eating: %d\n", ++time);
        race_delay();

        chop[3].V();
        chop[4].V();

        race_delay();
    }while(true);
}

```

● 实验结果展示：

3.1 运行一段时间后观察到每个哲学家都可以吃到食物

```
pholo1 is eating: 90
5242txc is eating: 91
pholo4 is eating: 90
pholo3 is eating: 90
pholo3 is eating: 91
pholo2 is eating: 92
ppholo1 is eating: 91
```

3.2 运行一段时间后观察到每个哲学家都可以吃到食物，没有出现死锁一直等待的情况，且第一个拿起筷子的哲学家可能会吃更多次

```
pholo4 is eating: 115
5242txc is eating: 175
pholo3 is eating: 113
pholo1 is eating: 136
5242txc is eating: 176
pholo2 is eating: 139
pholo4 is eating: 116
```

Section 5 实验总结与心得体会

实验总结：

本次实验通过初步了解自旋锁和信号量解决同步互斥问题以及如何解决“抽烟者问题”和“哲学家就餐”，深入探究了操作系统中并发同步的核心挑战。

1. **竞争态验证：**在不使用同步工具的情况下，通过人为引入 `race_delay()`，成功在 QEMU 输出中观测到了信号丢失、资源抢夺等逻辑冲突，证明了非原子操作在多线程环境下的不可靠性。
2. **信号量应用：**通过引入信号量机制，利用其 P/V 操作的原子性和阻塞唤醒功能，实现了线程间的有序同步。
3. **死锁预防：**在哲学家就餐实验中，通过观察“全左手持有”导致的系统挂起，验证了循环等待产生的死锁现象，并最终通过破坏死锁必要条件（改变特定线程获取资源的顺序）成功解决了该问题。

心得体会：

本次实验中我更深刻理解了 `i++` 在多线程环境下其实是复杂的“读-改-写”过程。此外并发能显著提升系统效率，但也带来了死锁问题。在解决哲学家死锁的过程中，我体会到算法设计对于系统稳定性的重要意义——不仅仅是让程序“跑起来”，更要让它在复杂的随机调度下依然“跑得稳”。

Section 6 附录：参考资料清单

1. 实验指导——[SYSU-2023-Spring-Operating-System: 中山大学 2023 学年春季操作系统课程 - Gitee.com](#)
2. 参考——[OS lab6 实现自旋锁，信号量 - RogersLuo](#)
3. 大模型——[Test-and-Set Assembly Logic Explained - Google Gemini](#)